

A.Tejaswini-192471043

Q101. Minimum k-Dominating Set Problem (NP-Complete Problem)

Determine a set of vertices such that every vertex in the graph is within distance k from at least one selected vertex. Clearly define constraints related to distance coverage and vertex selection. Analyze how constraints affect feasible dominating sets. Develop a backtracking-based solution to explore vertex subsets. Apply pruning to eliminate non-covering sets early. Justify correctness and explain the NP-Complete complexity.

Ans:

```
from collections import deque
```

```
def k_dominating_set(graph, k):
```

```
vertices = list(graph.keys())
```

```
all_vertices = set(vertices)
```

```
def bfs_cover(start):
```

$$\text{visited} = \{\text{start}\}$$

```
queue = deque([(start, 0)])
```

while queue:

```
node, distance = queue.popleft()
```

```

if distance == k:

```

continue

```
for neighbor in graph[node]:
```

```

if neighbor not in visited:

```

```
visited.add(neighbor)
```

```
queue.append((neighbor, distance + 1))
```

```
return visited
```

```
cover = {}
```

```
for vertex in vertices:
```

```
cover[vertex] = bfs_cover(vertex)
```

```
n = len(vertices)
```

```
suffix_cover = [set() for _ in range(n + 1)]
```

```
for i in range(n - 1, -1, -1):
```

```
suffix_cover[i] = suffix_cover[i + 1] | cover[vertices[i]]
```

```
best_set = None
```

```
def backtrack(index, selected, covered):
```

```
    nonlocal best_set
```

```
    if covered == all_vertices:
```

```
        if best_set is None or len(selected) < len(best_set):
```

```
            best_set = selected[:]
```

```
    return
```

```
    if index == n:
```

```
        return
```

```
    if best_set is not None and len(selected) >= len(best_set):
```

```
        return
```

```
if not all_vertices.issubset(covered | suffix_cover[index]):
```

```
    return
```

```
    vertex = vertices[index]
```

```
    # Choose the current vertex
```

```
    if not cover[vertex].issubset(covered):
```

```
        backtrack(
```

```
            index + 1,
```

```
            selected + [vertex],
```

```
            covered | cover[vertex]
```

```
        )
```

```
    # Do not choose the current vertex
```

```
    backtrack(index + 1, selected, covered)
```

```
    backtrack(0, [], set())
```

```
    return best_set
```

```
graph = {
```

```
    0: [1],
```

```
    1: [0, 2],
```

```
    2: [1, 3, 4],
```

```
    3: [2],
```

```
    4: [2]
```

```
}
```

```
k = 1
```

```
answer = k_dominating_set(graph, k)
```

```
print("Minimum", k, "-Dominating Set:", answer)
```

```
print("Size of Dominating Set:", len(answer))
```

Q102. Constraint-Based Virtual Machine Placement (NP-Hard Problem) Assign virtual machines to physical servers minimizing energy consumption while satisfying capacity constraints. Clearly define constraints such as CPU, memory, and network bandwidth. Analyze how constraints influence placement decisions. Develop a backtracking-based solution to explore feasible mappings. Apply pruning to eliminate overloaded configurations. Justify correctness and discuss NP-Hard complexity.

Ans:

```
def vm_placement(vms, servers):
    number_of_vms = len(vms)
    number_of_servers = len(servers)
    order = sorted(
        range(number_of_vms),
        key=lambda i: vms[i]["cpu"] + vms[i]["memory"] + vms[i]["bandwidth"],
        reverse=True
    )
    loads = []
    for _ in range(number_of_servers):
        loads.append({
            "cpu": 0,
            "memory": 0,
            "bandwidth": 0
        })
    placement = [None] * number_of_vms
    best_energy = float("inf")
    best_placement = None
    def server_energy(server_index):
        load = loads[server_index]
        server = servers[server_index]
        if load["cpu"] == 0 and load["memory"] == 0 and load["bandwidth"] == 0:
            return 0
        cpu_utilization = load["cpu"] / server["cpu"]
```

```

    energy = server["idle_power"] + cpu_utilization * (
        server["max_power"] - server["idle_power"]
    )
    return energy
def total_energy():
    total = 0
    for i in range(number_of_servers):
        total += server_energy(i)
    return total
def can_place(vm, server_index):
    server = servers[server_index]
    load = loads[server_index]
    if load["cpu"] + vm["cpu"] > server["cpu"]:
        return False
    if load["memory"] + vm["memory"] > server["memory"]:
        return False
    if load["bandwidth"] + vm["bandwidth"] > server["bandwidth"]:
        return False

    return True

def add_vm(vm, server_index):
    loads[server_index]["cpu"] += vm["cpu"]
    loads[server_index]["memory"] += vm["memory"]
    loads[server_index]["bandwidth"] += vm["bandwidth"]

def remove_vm(vm, server_index):
    loads[server_index]["cpu"] -= vm["cpu"]
    loads[server_index]["memory"] -= vm["memory"]
    loads[server_index]["bandwidth"] -= vm["bandwidth"]

```

```

def backtrack(position):
    nonlocal best_energy, best_placement

    if position == number_of_vms:
        energy = total_energy()

        if energy < best_energy:
            best_energy = energy
            best_placement = placement[:]

    return

current_energy = total_energy()

if current_energy >= best_energy:
    return

vm_index = order[position]
vm = vms[vm_index]

possible_servers = []

for server_index in range(number_of_servers):
    if can_place(vm, server_index):
        before_energy = server_energy(server_index)

        add_vm(vm, server_index)
        after_energy = server_energy(server_index)

```

```
remove_vm(vm, server_index)
```

```
extra_energy = after_energy - before_energy
```

```
possible_servers.append((extra_energy, server_index))
```

```
possible_servers.sort()
```

```
for extra_energy, server_index in possible_servers:
```

```
    add_vm(vm, server_index)
```

```
    placement[vm_index] = server_index
```

```
    backtrack(position + 1)
```

```
    placement[vm_index] = None
```

```
    remove_vm(vm, server_index)
```

```
backtrack(0)
```

```
return best_energy, best_placement
```

```
vms = [
```

```
    {"name": "VM1", "cpu": 2, "memory": 4, "bandwidth": 2},
```

```
    {"name": "VM2", "cpu": 4, "memory": 8, "bandwidth": 3},
```

```
    {"name": "VM3", "cpu": 1, "memory": 2, "bandwidth": 1},
```

```
    {"name": "VM4", "cpu": 3, "memory": 4, "bandwidth": 2}
```

```
]
```

```
servers = [
```

```

{
    "name": "Server1",
    "cpu": 6,
    "memory": 12,
    "bandwidth": 6,
    "idle_power": 100,
    "max_power": 200
},
{
    "name": "Server2",
    "cpu": 8,
    "memory": 16,
    "bandwidth": 8,
    "idle_power": 120,
    "max_power": 220
}
]

```

```

energy, placement = vm_placement(vms, servers)

```

```

if placement is None:

```

```

    print("No feasible VM placement found")

```

```

else:

```

```

    print("Minimum Energy Consumption:", energy)

```

```

    print("VM Placement:")

```

```

    for i in range(len(vms)):

```

```

        server_index = placement[i]

```

```

        print(vms[i]["name"], "->", servers[server_index]["name"])

```